

**Pokhara University
Gandaki College of Engineering and Science
Lamachaur-16, Kaski**



**Lab 2: Implementation of Source code management using git commands
through GitHub – Build, Breakpoints, Refactoring, Collaboration,
Copilot, IntelliSense**

Submitted By:

Aryan Giri
BESE2023
Roll no. 15

Submitted To:

Er. Rajendra Bahadur Thapa

OBJECTIVE:

This lab applies Git-based source code management practices within GitHub across five focus areas: build configuration, breakpoint-driven debugging, refactoring, collaborative workflows, and AI-assisted coding with Copilot and IntelliSense.

2a: Build – Release and Debug**AIM:**

To study and apply build configurations in software development, distinguishing between Debug and Release modes and examining how each affects application behaviour and performance.

THEORY:

A build configuration determines the way source code is compiled and linked to produce an executable. Software development typically relies on two primary configurations: Debug and Release.

Debug Build: retains complete symbolic debugging information, turns off compiler optimizations, and activates runtime checks. The resulting executable is larger and runs more slowly, but it is far easier to trace and inspect while developing.

Release Build: applies compiler optimizations and removes debugging symbols, yielding a smaller and faster executable appropriate for distribution to end users.

Within VS Code, these build configurations are handled through the `launch.json` and `tasks.json` files located inside the `.vscode` folder.

OBJECTIVES:

- Distinguish between Debug and Release build configurations.
- Set up a project with both Debug and Release build options in VS Code.
- Compile and execute the project under each configuration.

- Compare the resulting output, executable size, and performance across both builds.

ALGORITHM:

1. Set up a new project folder and open it within VS Code.
2. Write a sample program, such as a basic calculator in C++ or Node.js.
3. Create the file .vscode/tasks.json and define a Debug build task inside it.
4. Add a Release build task that includes the relevant optimization flags.
5. Execute the Debug build and observe its output.
6. Execute the Release build and compare executable size and run time against the Debug build.
7. Record the observations made during the comparison.

CODE:

Sample C++ program (main.cpp):

```
#include <iostream>
using namespace std; int
main() {
    int a=10,b=5;
    cout << "Sum: " << a + b << endl;
    cout<<"Product:"<<a*b<<endl; return
    0;
}
```

Build tasks (tasks.json):

```
{
  "version": "2.0.0",
  "tasks": [
    { "label": "BuildDebug", "type": "shell", "command": "g++-g-odebug_out main.cpp" },
    { "label": "BuildRelease", "type": "shell", "command": "g++-O2-orelease_out main.cpp" }
  ]
}
```

OUTPUT:

```
Sum:15
Product:50
```

- Debug binary: comparatively larger, and includes the symbol table.

- Release binary: comparatively smaller, executes faster, and excludes debug symbols.

RESULT:

Both the Debug and Release builds compiled and ran without issue. The Debug build kept its full set of symbol information, which is valuable for troubleshooting, whereas the Release build yielded an optimized binary suited to production use.

CONCLUSION:

This exercise illustrated why build configurations matter throughout the software development lifecycle: Debug builds facilitate development and testing, while Release builds are tuned for deployment to end users.

2b: Breakpoints**AIM:**

To study and apply breakpoints within VS Code for software debugging, covering how to set, manage, and use conditional breakpoints to examine program state while the program runs.

THEORY:

A breakpoint is a debugging mechanism that halts program execution at a chosen line, giving the developer the opportunity to examine variables, the call stack, and the flow of the program at that instant.

VS Code supports several types of breakpoints:

- Line Breakpoint: halts execution at a designated line number.
- Conditional Breakpoint: halts execution only once a specified condition evaluates to true, for example `i == 5`.
- Logpoint: writes a message to the console while allowing execution to continue uninterrupted.
- Exception Breakpoint: halts execution the moment an exception is thrown.

OBJECTIVES:

- Place a simple line breakpoint and observe the program pausing there.
- Examine variable values at the point where execution pauses.
- Apply conditional breakpoints during debugging.
- Make use of the Watch panel and Call Stack view while debugging.

ALGORITHM:

1. Open a JavaScript or Python project within VS Code.
2. Write a program containing a loop and a variable worth tracking.
3. Click within the gutter, to the left of the line number, to place a breakpoint.
4. Open the Run and Debug panel using Ctrl+Shift+D.
5. Begin the debugging session by pressing F5.
6. Check variable values through the Variables panel.
7. Add a conditional breakpoint via right-click on the gutter and selecting Edit Breakpoint.
8. Resume execution and confirm that the breakpoint triggers under the correct condition.

CODE:

Sample JavaScript program (debug_demo.js):

```
function calculateSum(n) {  
    let sum = 0;  
    for (let i = 1; i <= n; i++) {  
        sum += i; // Set breakpoint here  
    }  
    return sum;  
}  
console.log('Sum:', calculateSum(10));
```

launch.json configuration:

```
{  
  "version": "0.2.0",  
  "configurations": [  
    {  
      "type": "node",  
      "request": "launch",  
      "name": "Debug Demo",
```

```
    "program": "${workspaceFolder}/debug_demo.js"
  }
]
```

OUTPUT:

- Execution came to a stop at the breakpoint placed on the `sum += i` line.
- The Variables panel displayed the current values of `i` and `sum` on every iteration.
- The conditional breakpoint (`i === 5`) fired only once `i` reached the value 5.

Sum: 55

RESULT:

Breakpoints were configured and applied successfully to pause, examine, and trace the program's execution within VS Code.

CONCLUSION:

Employing VS Code's debugger together with line and conditional breakpoints allows developers to trace logic errors efficiently and examine runtime state, a practice that is essential to Agile's iterative style of development.

2c: Refactoring

AIM:

To study and apply code refactoring techniques through VS Code's built-in tools, with the aim of improving code readability, maintainability, and structure while leaving its external behaviour unchanged.

THEORY:

Refactoring refers to reorganizing existing code without changing what it outwardly does. Frequently used techniques include:

- **Rename Symbol:** renames a variable or function consistently across the entire codebase, invoked using F2 in VS Code.
- **Extract Function:** relocates a block of code into its own named function.

- Extract Variable: substitutes a repeated expression with a single named variable.
- Inline Variable: replaces a variable that is used only once with its direct value.

OBJECTIVES:

- Identify sections of code that would benefit from refactoring.
- Apply the rename, extract function, and extract variable refactorings using VS Code.
- Confirm that the program's behaviour is unaffected after refactoring.

ALGORITHM:

1. Open a JavaScript project in VS Code containing code that is poorly structured.
2. Locate a lengthy function that performs several unrelated tasks.
3. Highlight a block of code and apply Refactor > Extract Function.
4. Rename a poorly chosen variable name via F2 > Rename Symbol.
5. Execute the program both before and after refactoring to confirm the output stays identical.
6. Record the code structure both before and after the refactoring.

CODE:

Before refactoring:

```
function process(a, b) {  
  let x = a * a + b * b;  
  let y = Math.sqrt(x);  
  console.log('Result:', y);  
  let z = a + b;  
  console.log('Sum:', z);  
}
```

After refactoring:

```
function calculateHypotenuse(sideA, sideB) {  
  return Math.sqrt(sideA ** 2 + sideB ** 2);  
}  
function calculateSum(sideA, sideB) {  
  return sideA + sideB;  
}  
function process(sideA, sideB) {  
  console.log('Hypotenuse:', calculateHypotenuse(sideA, sideB));  
  console.log('Sum:', calculateSum(sideA, sideB));  
}
```

OUTPUT:

```
Hypotenuse: 5  
Sum: 7
```

- Readability of the code improved considerably.
- Every function now carries out a single, well-defined responsibility.

RESULT:

Refactoring was carried out successfully with VS Code's built-in tooling. The refactored version produced the same output as before, while being noticeably more readable and easier to maintain.

CONCLUSION:

The refactoring tools built into VS Code make it straightforward to safely rename, extract, and restructure code, which in turn reduces technical debt and supports the long-term health of a project within Agile development.

2d: Source Code Collaboration

AIM:

To put collaborative software development practices into effect using GitHub, encompassing feature branching, pull requests, and code review workflows of the kind used by Agile teams.

THEORY:

Source code collaboration makes it possible for several developers to work on a shared codebase at the same time. GitHub supports this through the following mechanisms:

- Branching: establishes an isolated space for developing new features without disturbing the main codebase.
- Pull Requests (PRs): formal proposals to merge a feature branch into main, which opens the door to code review.

- Code Review: team members examine proposed changes, leave feedback, and approve PRs prior to merging.
- Fork and PR Workflow: external contributors fork the repository and submit PRs, the standard model followed in open-source projects.

OBJECTIVES:

- Create Git branches and switch between them.
- Introduce changes on a feature branch and push them to GitHub.
- Open a Pull Request and walk through a simulated code review.
- Merge the feature branch into main once the review is complete.

ALGORITHM:

1. Clone the shared repository to the local machine.
2. Create a new feature branch using `git checkout -b feature/my-feature`.
3. Introduce the necessary changes and commit them.
4. Push the feature branch up to GitHub.
5. Open a Pull Request on GitHub with main as the target branch.
6. Request a review from a teammate and respond to any comments raised.
7. Merge the Pull Request once it has been approved.

CODE:

Create and switch to a feature branch:

```
git checkout -b feature/add-login
```

Stage, commit, and push:

```
git add .  
git commit -m "Add login functionality"  
git push origin feature/add-login
```

Merge after approval:

```
git checkout main  
git merge feature/add-login  
git push origin main
```

OUTPUT:

- The feature branch was created and pushed without issue.
- A Pull Request was opened on GitHub, with the changes clearly visible in the diff view.
- Review comments were added and subsequently resolved.
- The Pull Request was merged into main with no conflicts encountered.

RESULT:

Collaborative source code management through GitHub's branching and Pull Request workflow was implemented successfully.

CONCLUSION:

The Pull-Request-based workflow represents the industry norm across both open-source and professional software projects, allowing Agile teams to work in parallel while keeping the main branch clean.

2e: GitHub Copilot, Code Generation, and IntelliSense**AIM:**

To investigate and apply AI-assisted code generation with GitHub Copilot and IntelliSense inside VS Code, and to understand how these tools boost developer productivity within an Agile setting.

THEORY:

IntelliSense is the code-completion system built into VS Code. It offers real-time suggestions, encompassing autocompletion, parameter hints, quick documentation, and go-to-definition.

GitHub Copilot is an AI-driven code generation tool that relies on a large language model to propose complete lines, functions, or blocks of code, drawing on context and natural-language comments.

- IntelliSense: autocompletes variable names, function signatures, and imports while you type.
- GitHub Copilot: produces function bodies based on comments or partially written code.
- Context-Aware Suggestions: Copilot tailors its suggestions to the existing code style and imports already in use.

OBJECTIVES:

- Set up IntelliSense for a JavaScript project within VS Code.
- Install and enable GitHub Copilot inside VS Code.
- Use Copilot to produce functions directly from natural-language comments.
- Compare the completions offered by IntelliSense against the suggestions produced by Copilot.

ALGORITHM:

1. Open VS Code and create a new JavaScript file.
2. Watch IntelliSense trigger automatically while typing, or force it manually with Ctrl+Space.
3. Install the GitHub Copilot extension from the VS Code Marketplace.
4. Sign in using a GitHub account that has access to Copilot.
5. Write a comment describing the function to be generated.
6. Press Enter and observe Copilot's suggested code, shown as grey ghost text.
7. Accept the suggestion by pressing Tab.
8. Run the generated function and confirm that it behaves correctly.

CODE:

IntelliSense – typing Math. triggers suggestions:

```
const x = Math. // IntelliSense shows: abs, ceil, floor, round, sqrt...
```

GitHub Copilot – comment-driven code generation:

```
// Function to check if a number is prime
function isPrime(n) {
```

```
    if (n < 2) return false;
    for (let i = 2; i <= Math.sqrt(n); i++) {
        if (n % i === 0) return false;
    }
    return true;
} // Copilot generated this from the comment above
```

Testing the generated function:

```
console.log(isPrime(7)); // true
console.log(isPrime(10)); // false
```

OUTPUT:

- IntelliSense supplied real-time completions for Math methods as well as function parameters.
- Copilot produced a fully correct isPrime function from a single descriptive comment.

```
true
false
```

RESULT:

AI-assisted code generation through GitHub Copilot and IntelliSense was demonstrated successfully. Copilot produced correct implementations from natural-language comments, while IntelliSense delivered ongoing, context-aware completions.

CONCLUSION:

GitHub Copilot and IntelliSense noticeably speed up development by cutting down on repetitive typing and offering correct implementations. That said, any AI-generated code should be reviewed and tested thoroughly before it is committed.